MySQL Assignment 2 – Querying Data

Employee Database (Part 2)

Author: Aiswarya Krishna G | Tools: MySQL 8.0, MySQL Workbench

1. Objective

This assignment continues from Assignment 1 and uses the same employee database. The aim is to practise retrieving and analysing data with SELECT queries: removing duplicates, aliases, filtering, sorting, limiting rows, aggregate functions, grouping, HAVING, and joins.

2. Database Used

The employee database contains three tables, created with constraints in Assignment 1:

Table	Columns	Rows loaded
departments	department_id (PK), department_name	13
location	location_id (PK, auto-increment), location	4
employees	employee_id (PK), employee_name, gender, age, hire_date, designation, department_id (FK), location_id (FK), salary	30

3. Setup: Loading the Data

The tables are populated with the provided insert script. Parent tables are loaded first so that the foreign keys in employees are satisfied: departments, then location, then employees. The counts are verified afterwards.

```
USE employee;
SELECT COUNT(*) FROM departments;  -- 13
SELECT COUNT(*) FROM location;     -- 4
SELECT COUNT(*) FROM employees;    -- 30
```

4. Basic Queries

4.1 Distinct Values

DISTINCT removes duplicate values from the result. Here it lists each salary once.

```
SELECT DISTINCT salary FROM employees;
```

Result: 11 distinct salaries, from 45000 to 100000.

4.2 Alias (AS)

A column alias gives a column a new name in the output without changing the table.

```
SELECT age AS Employee_Age, salary AS Employee_Salary FROM employees;
```

Result: 30 rows with the headings Employee_Age and Employee_Salary.

4.3 WHERE Clause and Operators

Employees with a salary above 50000 who were hired before 2016-01-01. The AND operator requires both conditions to be true.

```
SELECT * FROM employees
WHERE salary > 50000 AND hire_date < '2016-01-01';
```

Result: 5 employees (IDs 5001, 5002, 5004, 5005, 5006). Employee 5003 is excluded because the salary is only 45000.

Finding the employee whose designation is missing, then filling it in with "Data Scientist". IS NULL is used because = NULL never matches. Safe update mode is switched off so the UPDATE can run without a key in the WHERE clause.

```
SELECT * FROM employees WHERE designation IS NULL;

SET SQL_SAFE_UPDATES = 0;

UPDATE employees
SET designation = 'Data Scientist'
WHERE designation IS NULL;

SELECT * FROM employees WHERE employee_id = 5004;
```

Result: Employee 5004 (Kiara Malhotra) is the only row with a missing designation, and it is updated to "Data Scientist" (1 row affected).

5. Sorting and Grouping Data

5.1 ORDER BY

Sorts by department_id in ascending order. Employees in the same department are then sorted by salary in descending order.

```
SELECT * FROM employees
ORDER BY department_id ASC, salary DESC;
```

5.2 LIMIT

The first 5 employees hired in 2018. YEAR() extracts the year, ORDER BY puts the earliest hire dates first, and LIMIT keeps only 5 rows.

```
SELECT * FROM employees
WHERE YEAR(hire_date) = 2018
ORDER BY hire_date
LIMIT 5;
```

Result: Employees 5019 to 5023. Employee 5024 was also hired in 2018 but is the sixth row, so LIMIT cuts it.

5.3 Aggregate Functions

Sum of all salaries in the Finance department. A join is needed because the department name is stored in the departments table.

```
SELECT SUM(e.salary) AS total_finance_salary
FROM employees e
JOIN departments d ON e.department_id = d.department_id
WHERE d.department_name = 'Finance';
```

Result: 170000 (Ishaan Kumar 85000 + Advik Kapoor 85000).

Minimum age among all employees:

```
SELECT MIN(age) AS min_age FROM employees;
```

Result: 25

5.4 GROUP BY

Maximum salary for each location:

```
SELECT l.location, MAX(e.salary) AS max_salary
FROM employees e
JOIN location l ON e.location_id = l.location_id
GROUP BY l.location;
```

location	max_salary
Chennai	95000
Bangalore	95000
Hyderabad	90000
Pune	100000

Average salary for each designation containing the word "Analyst". LIKE with % wildcards matches any text around the word.

```
SELECT designation, AVG(salary) AS avg_salary
FROM employees
WHERE designation LIKE '%Analyst%'
GROUP BY designation;
```

designation	avg_salary
Data Analyst	60000
Marketing Analyst	60000
Supply Chain Analyst	60000
Financial Analyst	85000
Finance Analyst	85000
Business Analyst	75000
Quality Assurance Analyst	80000

5.5 HAVING

Departments with fewer than 3 employees. HAVING filters groups after aggregation, whereas WHERE filters rows before grouping.

```
SELECT d.department_name, COUNT(e.employee_id) AS emp_count
FROM departments d
LEFT JOIN employees e ON d.department_id = e.department_id
GROUP BY d.department_id, d.department_name
HAVING COUNT(e.employee_id) < 3;
```

department_name	emp_count
Software Development	1
Product Management	2
Content Creation	2

department_name	emp_count
Finance	2
Research and Development	1
Business Development	2
Operations	1

Locations where the average age of female employees is below 30. WHERE keeps only female employees first, then HAVING checks the group average.

```
SELECT l.location, AVG(e.age) AS avg_age
FROM employees e
JOIN location l ON e.location_id = l.location_id
WHERE e.gender = 'F'
GROUP BY l.location
HAVING AVG(e.age) < 30;
```

location	avg_age
Chennai	26.75
Bangalore	27.33
Pune	29.20

Hyderabad is excluded because its female average age is 31.00.

6. Joins

6.1 Inner Join

Returns only rows that have a match in both tables: employee names, designations and department names.

```
SELECT e.employee_name, e.designation, d.department_name
FROM employees e
INNER JOIN departments d ON e.department_id = d.department_id;
```

Result: All 30 employees, since every employee belongs to a department.

6.2 Left Join

Lists all departments with the number of employees in each, including departments that have none. COUNT(e.employee_id) is used instead of COUNT(*) so an empty department shows 0, not 1.

```
SELECT d.department_name, COUNT(e.employee_id) AS total_employees
FROM departments d
LEFT JOIN employees e ON d.department_id = e.department_id
GROUP BY d.department_id, d.department_name;
```

department_name	total_employees
Software Development	1
Marketing	3
Data Science	3
Human Resources	3
Product Management	2

department_name	total_employees
Content Creation	2
Finance	2
Design	3
Research and Development	1
Customer Support	3
Business Development	2
IT	4
Operations	1

6.3 Right Join

Displays all locations with the names of employees assigned to each. A location with no employees would show NULL for employee_name.

```
SELECT l.location, e.employee_name
FROM employees e
RIGHT JOIN location l ON e.location_id = l.location_id;
```

Result: 30 rows. In this dataset every location has employees, so no NULL values appear.

7. Concepts Covered

Topic	Keyword(s) used	Purpose
Distinct values	DISTINCT	Remove duplicate values
Alias	AS	Rename columns in the output
Filtering	WHERE, AND, IS NULL, LIKE	Select rows by condition
Sorting	ORDER BY ASC / DESC	Order the result
Limiting	LIMIT	Restrict the number of rows
Aggregates	SUM, MIN, MAX, AVG, COUNT	Calculate on groups of rows
Grouping	GROUP BY, HAVING	Summarise data and filter groups
Joins	INNER, LEFT, RIGHT JOIN	Combine data from related tables

8. Key Learnings

- WHERE filters individual rows before grouping; HAVING filters groups after aggregation.
- Table aliases (e, d, l) make join queries shorter and avoid ambiguous column names.
- IS NULL must be used to find missing values; = NULL never matches.
- INNER JOIN keeps only matching rows, while LEFT and RIGHT JOIN also keep unmatched rows from one side.
- COUNT(column) ignores NULLs, which gives the correct 0 for departments with no employees.